Linux Shell Overview

1. Linux Shell

When you open a terminal in TryHackMe, you see a prompt like:

```
user@tryhackme:~$
```

Most distributions use Bash (Bourne Again Shell) by default.
The shell is where you type commands to interact with the system.

2. Current Directory

To see where you are:

```
pwd
```

Example output:
/home/user

This shows your current working directory.

3. Change Directory

To move to another directory:

```
cd Desktop
```

Now your prompt updates to show the new directory:
user@tryhackme:~/Desktop$

4. List Directory Contents

To see what files and folders are in your current directory:

```
ls
```

Example output:
Desktop Documents Downloads Music Pictures Public Templates Videos

5. Display File Contents

To read a file directly in the terminal:

```
cat filename.txt
```

Example output:

- this is a sample file
- this is the second line of the file

6. Search Inside Files: grep

grep searches for patterns or words in a file.

Syntax:

```
grep pattern filename
```

Example: Search for "THM" inside a file:

```
grep THM dictionary.txt
```

Type of Linux shells

Check & Change Shell

Current shell:

```
echo $SHELL
```

List available shells:

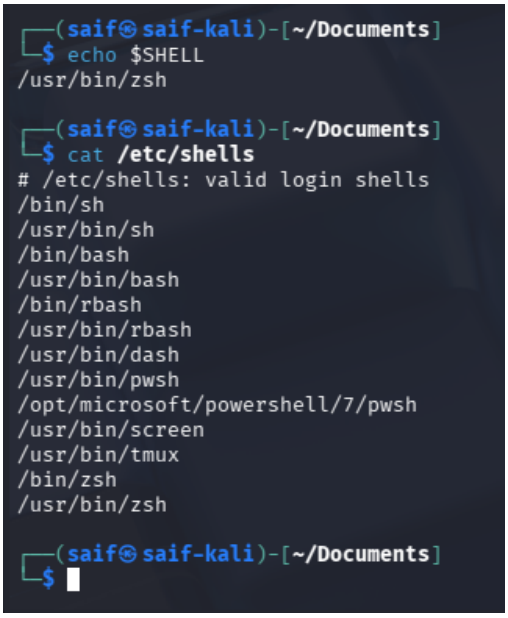
```
cat /etc/shells
```

Switch temporarily:

```
zsh   # or bash, fish, etc.
```

Set default shell permanently:

```
chsh -s /usr/bin/zsh
```



Linux Shell Scripting

What is a Shell Script?

- A shell script is a set of commands written in a text file, executed together to automate tasks.
- Saves time on repetitive tasks by running multiple commands at once.
- All Linux shells (Bash, Zsh, Fish) support scripting.
- Scripts are not limited to shells; any programming language can be used, but here we focus on bash scripting.

Creating a Shell Script

1. Open terminal and choose a shell (Bash recommended).
2. Create a file with .sh extension:

```
nano first_script.sh
```

3. Add the shebang at the top (defines interpreter):

```
#!/bin/bash
```

4. Write commands in the script. Example with variables and user input:

```
#!/bin/bash
echo "Hey, what’s your name?"
read name
echo "Welcome, $name"
```

5. Save and exit (CTRL+X → Y → ENTER).

Set Execution Permission

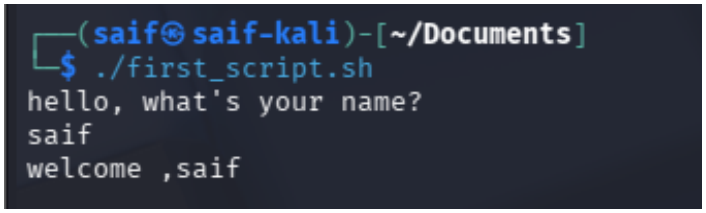
Before running a script, make it executable:

```
chmod +x first_script.sh
```

Execute the Script

Run the script in current directory using ./

```
./first_script.sh
```



Bash Loop

For Loop in Bash

The for loop runs a command block for each value in a list.

Basic Syntax

```
#!/bin/bash
for variable in list
do
commands
done
```

Example Script

Create a script:

```
nano loop_script.sh
```

Script code:

```
#!/bin/bash
for i in {1..10};
do
echo $i
done
```

Bash Conditional Statements

What are Conditional Statements?

- Conditional statements allow a script to make decisions.
- They execute different commands depending on whether a condition is true or false.

Example Use Cases

- Allow only certain users to access a secret
- Check if a file exists
- Verify user input
- Automate system checks

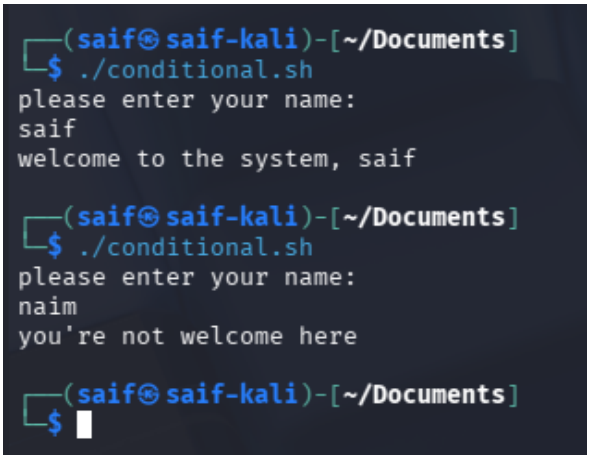
Basic if Statement Syntax

```
#!/bin/bash

if [ condition ]; then
    commands_if_true
else
    commands_if_false
fi
```

Explanation

1. if → starts the condition
2. [condition] → the test being checked
3. then → code executed if condition is true
4. else → code executed if condition is false
5. fi → ends the conditional block



Example Script

Create script:

```
nano conditional_script.sh
```

Script code:

```
#!/bin/bash
echo "Please enter your name first:"
read name

if [ "$name" = "Stewart" ]; then
    echo "Welcome Stewart! Here is the secret: THM_Script"
else
    echo "Sorry! You are not authorized to access the secret."
```

Comments in Bash Scripts

What is a Comment?

A comment is a note inside the script written for human understanding.

- Ignored during script execution
- Helps explain complex code

Comment Syntax

```
# This is a comment
```

Bash Script Example – Authentication System

This script demonstrates the use of three core scripting concepts:

- Variables
- Loops
- Conditional Statements

The goal is to authenticate a user before allowing access to a locker.

Problem Requirement

The script should ask the user for:

1. Username
2. Company name
3. PIN

Access is granted only if all values match:

Field	Correct Value
Username	John
Company Name	Tryhackme
PIN	7385

If the details are incorrect → Access denied.

Script Code

```
#!/bin/bash

# Defining variables
username=""
companyname=""
pin=""

# Loop to collect user inputs
for i in {1..3}; do

    if [ "$i" -eq 1 ]; then
        echo "Enter your Username:"
        read username

    elif [ "$i" -eq 2 ]; then
        echo "Enter your Company name:"
        read companyname

    else
        echo "Enter your PIN:"
        read pin
    fi

done

# Authentication check
if [ "$username" = "John" ] && [ "$companyname" = "Tryhackme" ] && [ "$pin" = "7385" ]; then
    echo "Authentication Successful. You can now access your locker, John."
else
    echo "Authentication Denied!!"
fi
```